ThreatLens

Security Event Monitoring and Threat Detection System

CS-232 DBMS | Mr. Said Nabi | GIKI

Usman Azhar (2024653) | Fatima Ali (2024173) | Muhammad Abdullah (2024321)

25-Day Complete Workflow Document

1. What Is This Project

ThreatLens is a security monitoring system built as a database-driven application. It logs every action users take on a computer system, like logins, file access, and data exports, and then uses SQL queries to automatically detect if something suspicious is happening.

Think of it like a CCTV system but for computer activity. Instead of recording video, it records user actions in a database and raises alerts automatically when it detects patterns like someone trying to hack in.

The most important thing to understand about this project:

- The intelligence of this system lives in SQL queries, not in complex code.
- When someone tries to hack in with 5 wrong passwords in 15 minutes, a SQL query catches it.
- When a login comes from a known bad IP address, a SQL query catches it.
- This is why it is perfect for a DBMS course. The database does the actual work.

This system is called a SIEM (Security Information and Event Management) system. Real companies like banks and hospitals use systems exactly like this. You are building a student version of that.

2. The Database — 8 Tables and What They Store

The entire project is built on 8 tables in PostgreSQL. Every feature, every detection, every dashboard page reads from or writes to these tables. Understanding these 8 tables is the foundation of everything else.

Table 1 — Organizations

Who owns this: Abdullah

What it stores: The different companies or institutions using ThreatLens. For example, a bank, a hospital, or a university can all use the same ThreatLens system but their data stays separate.

Column	Type	What It Means
org_id	SERIAL (Primary Key)	Auto-generated unique number for each organization
org_name	VARCHAR(255)	Name of the organization, e.g. GIKI, Allied Bank
created_at	TIMESTAMP	When this organization was added to the system

Table 2 — Roles

Who owns this: Fatima

What it stores: The permission levels that exist in the system. There are three roles: Admin (can do everything), Analyst (can view and investigate alerts), and Viewer (can only view events).

Column	Type	What It Means
role_id	SERIAL (Primary Key)	Auto-generated unique number for each role
role_name	VARCHAR(50)	The name: Admin, Analyst, or Viewer
permissions	TEXT	Description of what this role is allowed to do

Table 3 — Users

Who owns this: Fatima

What it stores: Everyone who can log into the monitored systems. Each user belongs to one organization and has one role.

Column	Type	What It Means
user_id	SERIAL (Primary Key)	Auto-generated unique number for each user
org_id	INT (Foreign Key → Organizations)	Which organization this user belongs to
role_id	INT (Foreign Key → Roles)	What permission level this user has
username	VARCHAR(100)	Their login name

Column	Type	What It Means
email	VARCHAR(255)	Their email address
password_hash	VARCHAR(255)	Their password stored as a hash, never plain text
created_at	TIMESTAMP	When this user account was created

Table 4 — Assets

Who owns this: Abdullah

What it stores: The systems and applications being monitored. An asset could be a Login Portal, an Admin Panel, a File Server, or a Database. Events happen on assets.

Column	Type	What It Means
asset_id	SERIAL (Primary Key)	Auto-generated unique number for each asset
org_id	INT (Foreign Key → Organizations)	Which organization this asset belongs to
asset_name	VARCHAR(255)	Name of the system, e.g. Login Portal
asset_type	VARCHAR(100)	Type of system, e.g. Web Application, Database

Table 5 — Sessions

Who owns this: Fatima

What it stores: Every time a user logs in, a session row is created. When they log out, the session is closed. This lets the system detect things like the same user being logged in from two different locations at the same time.

Column	Type	What It Means
session_id	SERIAL (Primary Key)	Auto-generated unique number for each session
user_id	INT (Foreign Key → Users)	Which user started this session
org_id	INT (Foreign Key → Organizations)	Which organization this session belongs to
session_token	VARCHAR(256)	A secret code that proves the session is valid
ip_address	INET	The IP address where the login happened
device_info	VARCHAR(255)	Browser and OS info, e.g. Chrome on Windows 11
login_time	TIMESTAMP	When the session started
last_active	TIMESTAMP	Last time the user did anything in this session
logout_time	TIMESTAMP	When the session ended. NULL means still active.

Column	Type	What It Means
status	VARCHAR(50)	Active, Expired, Terminated, or Suspicious
is_flagged	BOOLEAN	True if this session triggered a security alert

Table 6 — Events

Who owns this: Usman

What it stores: Every single action that happens in the monitored systems. A login attempt, a file download, accessing the admin panel — all of these become rows in this table. This is the most important table in the whole project. All threat detection queries run on this table.

Column	Type	What It Means
event_id	SERIAL (Primary Key)	Auto-generated unique number for each event
user_id	INT (Foreign Key → Users)	Which user performed this action
asset_id	INT (Foreign Key → Assets)	Which system this action happened on
session_id	INT (Foreign Key → Sessions)	Which active session this happened in
timestamp	TIMESTAMP	Exactly when this event happened
event_type	VARCHAR(100)	What happened: login_success, login_failed, file_download, etc.
ip_address	INET	IP address from where the action was done
success	BOOLEAN	True if the action succeeded, False if it failed
metadata	TEXT	Any extra details about the event in simple text

Table 7 — Alerts

Who owns this: Usman

What it stores: When a threat detection query finds something suspicious, an alert is created automatically. Each alert is linked to the event that triggered it.

Column	Type	What It Means
alert_id	SERIAL (Primary Key)	Auto-generated unique number for each alert
event_id	INT (Foreign Key → Events)	Which event triggered this alert
alert_type	VARCHAR(100)	What kind of threat: brute_force, suspicious_ip, etc.
severity	VARCHAR(50)	How serious: Low, Medium, High, or Critical
status	VARCHAR(50)	Open, Investigating, or Resolved

Column	Type	What It Means
created_at	TIMESTAMP	When this alert was created

Table 8 — Threat_Intelligence

Who owns this: Abdullah

What it stores: A list of known bad IP addresses and attack patterns. When a user logs in, their IP is checked against this table. If it matches, an alert is raised immediately.

Column	Type	What It Means
threat_id	SERIAL (Primary Key)	Auto-generated unique number for each threat entry
threat_type	VARCHAR(100)	Type of threat: malicious_ip, known_attacker, tor_exit_node
value	VARCHAR(255)	The actual IP address or domain that is known to be bad
last_updated	TIMESTAMP	When this entry was last verified or updated

How the Tables Connect to Each Other

This is called referential integrity. You must understand this before writing any SQL.

- Organizations is the top-level table. Every other table eventually connects back to it.
- One Organization has many Users. One Organization has many Assets.
- One User has one Role. One User has many Sessions. One User has many Events.
- One Asset has many Events. One Event can have one Alert.
- Threat_Intelligence is standalone. It is just a lookup table of bad IPs.

INSERT order — always follow this sequence when adding data:

1. Organizations first
2. Roles second
3. Users third (needs org_id and role_id to exist first)
4. Assets fourth (needs org_id to exist first)
5. Sessions fifth (needs user_id to exist first)
6. Events sixth (needs user_id, asset_id, session_id to exist first)
7. Alerts last (needs event_id to exist first)
8. Threat_Intelligence any time — it has no foreign keys

Never reverse this order. PostgreSQL will reject the insert with a foreign key error.

3. The 5 Threat Detections — The Core of ThreatLens

These are the 5 SQL queries that make ThreatLens actually do something useful. These are what your instructor wants to see. They are all owned by Usman and Abdullah.

Detection 1 — Brute Force Attack (Usman)

What it catches: Someone trying many wrong passwords to break into an account.

Logic: Find any user who has 5 or more failed login events in the last 15 minutes.

SQL:

```
SELECT user_id, COUNT(*) AS failed_attempts
FROM events
WHERE event_type = 'login_failed'
AND timestamp > NOW() - INTERVAL '15 minutes'
GROUP BY user_id
HAVING COUNT(*) >= 5;
```

DBMS concepts shown: GROUP BY, HAVING, aggregate function COUNT, interval filtering.

Detection 2 — Suspicious IP Login (Abdullah)

What it catches: A login from an IP address that is in the Threat_Intelligence table.

Logic: Find events where the ip_address matches any value in Threat_Intelligence.

SQL:

```
SELECT e.event_id, e.user_id, e.ip_address, e.timestamp
FROM events e
JOIN threat_intelligence ti ON e.ip_address::text = ti.value
WHERE e.event_type = 'login_success';
```

DBMS concepts shown: JOIN between two tables, type casting, filtering.

Detection 3 — Off-Hours Login (Usman)

What it catches: Someone logging in between 2am and 5am, which is unusual and suspicious.

SQL:

```
SELECT user_id, timestamp, ip_address
FROM events
WHERE event_type = 'login_success'
AND EXTRACT(HOUR FROM timestamp) BETWEEN 2 AND 5;
```

DBMS concepts shown: EXTRACT function, BETWEEN operator, time-based filtering.

Detection 4 — Privilege Escalation (Abdullah)

What it catches: A regular user trying to access the Admin Panel or other assets they are not allowed to use.

Logic: Find events where a non-admin user accessed an asset of type Admin_Panel.

SQL:

```
SELECT e.user_id, u.username, a.asset_name, e.timestamp
FROM events e
JOIN users u ON e.user_id = u.user_id
JOIN roles r ON u.role_id = r.role_id
JOIN assets a ON e.asset_id = a.asset_id
WHERE a.asset_type = 'Admin_Panel' AND r.role_name != 'Admin';
```

DBMS concepts shown: Multiple JOINs across 4 tables, filtering on joined columns.

Detection 5 — Concurrent Sessions (Fatima)

What it catches: The same user logged in from two different IP addresses at the same time. This suggests their account was stolen.

SQL:

```
SELECT user_id, COUNT(DISTINCT ip_address) AS locations
FROM sessions
WHERE status = 'Active'
GROUP BY user_id
HAVING COUNT(DISTINCT ip_address) > 1;
```

DBMS concepts shown: COUNT DISTINCT, GROUP BY, HAVING, filtering active sessions.

4. How the System Works — Step by Step Flow

This is what happens when someone uses ThreatLens from start to finish.

9. A user opens the login page in their browser.
10. They type their username and password and click Login.
11. The frontend sends this to the Flask backend via an API call.
12. Flask checks the Users table in PostgreSQL to verify the username and password.
13. If correct, Flask creates a new row in the Sessions table and returns a session token.
14. At the same time, Flask inserts a row in the Events table with event_type = login_success.
15. If the password was wrong, Flask inserts event_type = login_failed instead.
16. Every 60 seconds, a background check runs the brute force detection query.
17. If 5 or more failed logins are found for one user in 15 minutes, an Alert row is inserted automatically by a database trigger.
18. The admin opens the dashboard, which calls the Alerts API, which queries the Alerts table.
19. The alert appears on screen showing which user triggered it, from which IP, at what time.
20. The admin can change the alert status from Open to Investigating to Resolved.

The three layers and what they do:

Layer	Technology	What It Does
Database	PostgreSQL	Stores all data. Runs threat detection queries. Has triggers for auto-alerts. This is where the real intelligence lives.
Backend	Python + Flask	Receives requests from the frontend. Runs SQL queries. Returns results as JSON. About 200 lines of code total.
Frontend	HTML + JavaScript	Shows the dashboard. Calls the backend APIs. Displays events, alerts, charts. Does not contain any logic.

5. Work Division — Who Does What

Each person owns one feature from top to bottom. This means each person designs the database table, writes the SQL queries, builds the API, and builds the dashboard page for their feature. Nobody is only doing the database. Nobody is only doing the frontend. Everyone touches all three layers.

Usman — Events and Alerts

Database tables: Events, Alerts

SQL queries to write:

- Brute force detection query (5 failed logins in 15 minutes)
- Off-hours login query (logins between 2am and 5am)
- Indexes on Events table: timestamp, user_id, ip_address, org_id
- Database trigger: automatically insert into Alerts when brute force is detected
- Stored procedures: convert the 2 detection queries into stored procedures
- ACID transaction demo: wrap event insert + alert insert in one BEGIN/COMMIT block
- EXPLAIN query: run EXPLAIN on the brute force query and write down what it says

API endpoints to build:

- POST /api/events — receives a new event from the frontend and inserts it into the Events table
- GET /api/events — returns a list of all events, supports filtering by date and user
- GET /api/alerts — returns all alerts with their severity and status
- PATCH /api/alerts/:id — updates an alert status to Investigating or Resolved

Dashboard pages to build:

- Events Log page — a table showing all events with columns for user, time, type, IP, success/fail. Has a filter by date.
- Alerts Panel page — a table showing all alerts with severity color coding. Allows status change.

Fatima — Users, Roles, and Sessions

Database tables: Users, Roles, Sessions

SQL queries to write:

- Concurrent session detection query (same user, 2 different IPs active at same time)
- Long session query (sessions active for more than 8 hours)
- RBAC query: return only data that belongs to the logged-in user's organization
- Session expiry: mark sessions as Expired when last_active is older than 30 minutes

API endpoints to build:

- POST /api/login — verify username and password, create a session row, return session token
- POST /api/logout — find the session by token, set logout_time and status to Terminated

- GET /api/sessions — return all currently active sessions
- GET /api/users — return all users for the admin dashboard
- Middleware: check session token on every API request to verify the user is logged in
- RBAC check: check the user's role before allowing access to admin-only endpoints
- Error handling: return clear error messages when something goes wrong in any endpoint

Dashboard pages to build:

- Login page — a simple form with username and password that calls the login API
- Sessions Monitor page — shows all active sessions. Highlights flagged ones.
- Simulate Attack button — sends 6 rapid failed login requests to trigger the brute force detection

Abdullah — Organizations, Assets, and Threat Intelligence

Database tables: Organizations, Assets, Threat_Intelligence

SQL queries to write:

- Suspicious IP lookup query (JOIN events with threat_intelligence on ip_address)
- Privilege escalation query (non-admin users accessing Admin_Panel assets)
- Statistics query: events per day for the last 7 days using GROUP BY DATE(timestamp)
- Statistics query: top 5 users with the most events this week
- Statistics query: count of alerts grouped by severity

API endpoints to build:

- GET /api/organizations — return all organizations
- GET /api/assets — return all assets for a given organization
- GET /api/threat-intelligence — return all known threat IPs
- GET /api/stats/events-per-day — return event counts grouped by day for the chart
- GET /api/stats/alerts-by-severity — return alert counts grouped by severity

Dashboard pages to build:

- Statistics page — shows charts using Chart.js: events per day bar chart, alerts by severity pie chart, top users table
- Organizations and Assets page — shows list of organizations and their assets
- SQL injection test — try typing SQL like ' OR 1=1 -- into the login form and confirm it is blocked

Shared Tasks — All Three Together

Task	When	How
ER Diagram	Day 3 to 4	Usman draws it on draw.io. All three review it together and agree before Day 5.
Normalization document	Day 5 to 6	Each person writes normalization steps for their own tables. Usman combines into one document.

Task	When	How
Schema review	Day 6	All three read each other's CREATE TABLE SQL before running it. Catch mistakes together.
Seed data	Day 9	Each person inserts realistic fake data for their own tables.
Integration day	Day 15	All three connect their work together. Switch to Usman's shared database. Test the full system.
Simulate Attack demo	Day 19	Run the full demo together: simulate brute force, watch alert appear, check dashboard.
Final report	Days 21 to 24	Each person writes the section about their own feature. Usman merges and formats.
Presentation rehearsal	Day 25	Each person presents their own feature. Practice the live demo once.

6. The 25-Day Plan — Day by Day

Every day has specific tasks. A task should not take more than 2 to 3 hours. If it takes longer, ask for help in the WhatsApp group immediately.

Week 1 — Days 1 to 5: Setup and Database Design

Goal: By end of Day 5, all 8 tables are created and running in PostgreSQL.

Day	Task	Who
1	Create JIRA account, create project ThreatLens, create 6 Epics, invite Fatima and Abdullah	Usman
1	Join JIRA, read this entire document, write down any questions	Fatima, Abdullah
1	Create GitHub repository, set up the folder structure (database/, backend/, frontend/), push first commit	Abdullah
1	Everyone clones the GitHub repo to their laptop	All Three
2	Add all tasks from this document into JIRA backlog with correct assignees and Epics	All Three
2	Install all tools: PostgreSQL, pgAdmin, Python, Flask, VS Code, Postman, Git	All Three
2	Test that PostgreSQL is working by creating a test database in pgAdmin	All Three
2	Install psycopg2: open terminal, type: pip install psycopg2-binary	All Three
3	Draw the ER diagram on draw.io showing all 8 entities and all relationships with cardinality labels	Usman
3	Review the ER diagram, check if relationships make sense, send feedback to Usman	Fatima, Abdullah
3	Research Flask basics: what is a route, what is a POST request, write a hello world Flask app	Fatima
3	Make rough wireframe sketches of all dashboard pages on paper or any drawing tool	Abdullah
4	Finalize ER diagram after feedback, share the final version in WhatsApp group	Usman
4	Write CREATE TABLE SQL for Events and Alerts with all columns, types, and foreign keys	Usman
4	Write CREATE TABLE SQL for Users, Roles, and Sessions with all columns, types, and foreign keys	Fatima
4	Write CREATE TABLE SQL for Organizations, Assets, and Threat_Intelligence	Abdullah
5	Share your SQL files with the group, everyone reads each other's SQL and checks for mistakes	All Three

Day	Task	Who
5	Run all 8 CREATE TABLE statements in pgAdmin and confirm all tables appear	All Three
5	Write normalization steps for your own tables: explain why they are in 3NF	All Three

Week 2 — Days 6 to 10: Queries, Triggers, and APIs

Goal: By end of Day 10, all 5 threat detection queries work and all API endpoints return correct data from the database.

Day	Task	Who
6	Add indexes on Events table: CREATE INDEX on timestamp, user_id, ip_address, org_id	Usman
6	Write and test the brute force detection query in pgAdmin	Usman
6	Write and test the off-hours login query in pgAdmin	Usman
6	Set up Flask app skeleton: create app.py, create db.py with PostgreSQL connection, test connection works	Fatima
6	Write and test the concurrent session detection query in pgAdmin	Fatima
6	Write and test the suspicious IP lookup query in pgAdmin	Abdullah
6	Write and test the privilege escalation query in pgAdmin	Abdullah
7	Write and test the statistics query for events per day using GROUP BY DATE(timestamp)	Abdullah
7	Write the database trigger for brute force auto-alert. This needs a TRIGGER FUNCTION first, then the TRIGGER.	Usman
7	Build POST /api/events endpoint: receive event data, insert into Events table, return success	Usman
7	Build POST /api/login endpoint: check username and password, create session, return token	Fatima
7	Build POST /api/logout endpoint: update session status to Terminated	Fatima
7	Build GET /api/organizations endpoint: return all organizations	Abdullah
8	Build GET /api/events endpoint: return events, allow filter by user_id and date	Usman
8	Build GET /api/alerts endpoint: return all alerts with event details using JOIN	Usman
8	Build GET /api/sessions endpoint: return all active sessions	Fatima
8	Build GET /api/users endpoint: return all users for the admin	Fatima
8	Build GET /api/assets endpoint: return assets for a given org_id	Abdullah
8	Build GET /api/threat-intelligence endpoint: return all threat entries	Abdullah

Day	Task	Who
9	Build GET /api/stats/events-per-day and GET /api/stats/alerts-by-severity	Abdullah
9	Build PATCH /api/alerts/:id endpoint: update alert status	Usman
9	Add session token middleware: check every request has a valid session token	Fatima
9	Add RBAC check: block non-admin users from admin-only routes	Fatima
9	Insert seed data for your own tables: at least 2 organizations, 5 users, 3 assets, 20 events, 5 alerts, 10 threat IPs	All Three
10	Test all your API endpoints in Postman. Make sure each one returns the correct data.	All Three
10	Convert the brute force query and off-hours query into PostgreSQL stored procedures	Usman
10	Write the ACID transaction demo: BEGIN; INSERT INTO events; INSERT INTO alerts; COMMIT; — run it and show it works	Usman
10	Add error handling to all endpoints: return a clear message if the database is down or input is wrong	Fatima
10	Push all code to GitHub	All Three

Week 3 — Days 11 to 15: Dashboard and Integration

Goal: By end of Day 15, the full system is connected. Database, backend, and frontend all work together.

Day	Task	Who
11	Build Events Log page in HTML: a table that fetches from GET /api/events and displays all events	Usman
11	Build Alerts Panel page: fetches alerts, shows severity, has a dropdown to change status	Usman
11	Build Login page: a form with username and password, calls POST /api/login, stores token in browser	Fatima
11	Build Sessions Monitor page: fetches active sessions, highlights flagged ones	Fatima
11	Build Statistics page: fetches stats APIs, draws bar chart and pie chart using Chart.js	Abdullah
11	Build Organizations and Assets page: shows list of orgs and their assets	Abdullah
12	Add date filter to Events Log page	Usman
12	Build Simulate Attack button: when clicked, sends 6 POST /api/events requests with event_type=login_failed	Fatima
12	Add navigation bar linking all pages together	Abdullah

Day	Task	Who
12	Run EXPLAIN on your slowest query and write down what the output says in your notes	All Three
13	Review all pages, fix any broken buttons or missing data, make the layout readable	All Three
13	Test that org_id filtering works: one organization cannot see another organization's data	Fatima
14	Full system walkthrough: Usman opens events page, Fatima opens sessions page, Abdullah opens stats page. All should show real data.	All Three
15	INTEGRATION DAY. Switch db.py on Fatima and Abdullah's laptops to point to Usman's PostgreSQL. Change localhost to Usman's IP.	All Three
15	Test the full demo flow: login as a user, log some events, click Simulate Attack, watch the alert appear, view it on the alerts panel	All Three
15	Fix any connection or data errors that appear during integration	All Three

Week 4 — Days 16 to 20: Testing and Fixing

Goal: By end of Day 20, every single feature is tested and confirmed working. No known bugs.

Day	Task	Who
16	Test all Events and Alerts endpoints in Postman with correct inputs and wrong inputs	Usman
16	Run the brute force query with 1000+ fake failed events inserted. Confirm it returns the right users.	Usman
16	Test full login and logout flow: login, check session appears, logout, check session is terminated	Fatima
16	Test that a user from Organization A cannot see events from Organization B	Fatima
16	Test all dashboard pages in the browser. Click every button. Check every table shows data.	Abdullah
16	Test SQL injection: in the login form type: ' OR 1=1 -- and confirm login is rejected	Abdullah
17	Fix all bugs found on Day 16. Each person fixes their own code.	All Three
17	Test concurrent session detection: log in as the same user from two different browsers, run the query, confirm it flags the user	Fatima
18	Full brute force demo test: click Simulate Attack 3 times fast, confirm alert is auto-created by trigger, confirm it appears on dashboard	All Three
18	Concurrency test: Fatima and Abdullah both send events to the API at the exact same time. Check no data is lost.	Fatima, Abdullah

Day	Task	Who
19	Practice the full demo from start to finish without stopping: login, simulate attack, show alert, show statistics	All Three
20	Buffer day. Fix anything that is still broken. Polish the UI. Clean up the code.	All Three

Days 21 to 25 — Report and Presentation

Day	Task	Who
21	Write report section: Introduction, Project Overview, ER Diagram with explanation, all 8 tables with schema	Usman
22	Write report section: Sessions entity explanation, Normalization steps for all tables, Stored procedures, ACID demo	Fatima
23	Write report section: Dashboard screenshots, Testing results and what was tested, Tools used and why	Abdullah
24	All three merge the three sections into one document, add cover page, table of contents, screenshots	All Three
25	Make 10 to 12 presentation slides. Each person presents their own section. Run the live demo once from start to finish.	All Three

7. How to Share One Database Across All Three Laptops

You do not need any cloud service. You use the university WiFi and point everyone to Usman's laptop.

Part 1 — Days 1 to 14: Everyone Works Locally

For the first two weeks, each person works on their own localhost PostgreSQL. You are all building independently so there is no need to connect yet. This avoids all network complexity while you are still learning.

Part 2 — Day 15 Onwards: Switch to Shared Database

On Day 15, Usman configures his PostgreSQL to accept outside connections. This is two small file changes.

Step 1 — Usman opens the file pg_hba.conf on his laptop:

Location on Windows: C:\Program Files\PostgreSQL\16\data\pg_hba.conf

Add this line at the very bottom of the file:

```
host    all    all    0.0.0.0/0    md5
```

Step 2 — Usman opens the file postgresql.conf in the same folder and finds this line:

```
#listen_addresses = 'localhost'
```

Change it to:

```
listen_addresses = '*'
```

Step 3 — Usman restarts PostgreSQL:

On Windows: open the Services app, find PostgreSQL, right click, click Restart.

Step 4 — Usman finds his IP address on the university WiFi:

Open Command Prompt and type: ipconfig

Find the line that says IPv4 Address. It will look like 192.168.1.45. Share this number in WhatsApp.

Step 5 — Fatima and Abdullah change one line in their db.py file:

```
# Before:
conn = psycopg2.connect(host='localhost', database='threatlens',
user='postgres', password='yourpassword')

# After (replace with Usman's actual IP):
conn = psycopg2.connect(host='192.168.1.45', database='threatlens',
user='postgres', password='yourpassword')
```

That is the only change. Everything else works exactly the same as localhost.

Important rules for the shared database period:

- Always be on the same WiFi network as Usman when working on the shared database.
- If working from home, Usman shares his phone hotspot and everyone connects to that instead.
- Never drop or delete another person's table. If you need to change your table structure, tell the group first.
- Do not change column names after Day 6. Other people's code depends on your column names.

8. JIRA — How to Use It From Zero

JIRA is a free tool to track who is doing what and whether tasks are done. You will use it every day.

First Time Setup — Usman Does This on Day 1

21. Go to atlassian.com/software/jira and click Get it free.
22. Sign up with your university email.
23. When asked for template, choose Scrum.
24. Set project name to ThreatLens and key to TL.
25. Click Project Settings then People, then invite Fatima and Abdullah by their emails.
26. Create 6 Epics with these names: E1 Setup and Planning, E2 Database Design, E3 Backend and Queries, E4 Dashboard, E5 Testing, E6 Report and Presentation.

How to Create a Task

27. Click the big blue Create button at the top.
28. Issue Type: Task.
29. Summary: write the task name, for example: Write brute force detection query.
30. Epic Link: pick which Epic this belongs to.
31. Assignee: pick the person who will do it.
32. Priority: High, Medium, or Low.
33. Click Create.

Daily Habit — Takes 2 Minutes

- Every morning: open JIRA Board, drag your task for today from To Do to In Progress.
- When you finish a task: drag it to Done.
- If you are stuck: add a comment to the task explaining what the problem is.
- Every Sunday: 15 minute team call to review the week and plan the next sprint.

Task Naming Format — Use This Always

Format	Example
[DB] task name	[DB] Write brute force detection query
[BE] task name	[BE] Build POST /api/events endpoint
[FE] task name	[FE] Build Events Log dashboard page
[TEST] task name	[TEST] Test login and logout flow
[DOC] task name	[DOC] Write normalization steps for Sessions table

9. Tools — What to Install and How to Use Them

Tool	Where to Get It	What It Is For
PostgreSQL	postgresql.org/download	The database where all 8 tables live
pgAdmin 4	pgadmin.org	A visual app to open PostgreSQL, see tables, and run SQL queries
Python 3	python.org/downloads	The programming language for writing the backend
Flask	pip install flask	Turns Python into a web server that can receive API requests
psycopg2	pip install psycopg2-binary	Connects Python and Flask to the PostgreSQL database
VS Code	code.visualstudio.com	The text editor where everyone writes code
Git	git-scm.com	Saves your code and lets you share it with the team
GitHub	github.com	The website where the shared Git repository lives
Postman	postman.com/downloads	A tool to test API endpoints before the frontend is built
draw.io	app.diagrams.net	A free tool to draw the ER diagram
Chart.js	chartjs.org — use via CDN	A JavaScript library to draw charts on the statistics page
JIRA	atlassian.com/jira	Task tracking and sprint planning for the team

VS Code extensions to install (everyone):

- Python by Microsoft — enables Python syntax highlighting and autocomplete
- PostgreSQL by Chris Kolkman — lets you run SQL queries directly inside VS Code
- Prettier — automatically formats your code to look clean
- GitLens — shows who wrote each line of code and when

10. GitHub — How to Use It as a Team

GitHub is how you share code without emailing files. Every person works on their own laptop and pushes their work to GitHub. Others pull it to get the latest version.

Folder Structure — Set This Up on Day 1

```
threatlens/
    database/
        schema.sql          (all 8 CREATE TABLE statements)
```

```
        seed_data.sql       (fake test data for all tables)
        queries.sql         (all 5 threat detection queries)
        triggers.sql        (brute force auto-alert trigger)
    backend/
        app.py              (main Flask application)
        db.py               (PostgreSQL connection)
        routes/             (one file per feature: events.py, sessions.py,
orgs.py)
    frontend/
        index.html          (main page with navigation)
        pages/              (events.html, alerts.html, sessions.html,
stats.html)
```

The 4 Git commands everyone needs to know:

Command	What It Does	When to Use It
git pull	Downloads the latest code from GitHub to your laptop	Every morning before you start working
git add .	Marks all your changed files ready to save	After you finish a task
git commit -m "message"	Saves your changes with a description	After git add, write what you did
git push	Uploads your saved changes to GitHub	After git commit, so others can see your work

Commit message format:

- [DB] Write brute force detection query
- [BE] Build POST /api/events endpoint
- [FE] Build Events Log page
- [TEST] Test login flow end to end

11. Difficulties You Will Face and How to Handle Them

These are real problems that will happen. Being prepared means they will not slow you down.

Difficulty 1 — Foreign Key Errors When Inserting Data

What happens: You try to insert an event but PostgreSQL says there is no user with that user_id.

Why it happens: You are inserting in the wrong order. Events need users to exist first.

Fix: Always follow the insert order from Section 2. Organizations first, Roles second, Users third, Assets fourth, Sessions fifth, Events sixth, Alerts last.

Difficulty 2 — The Brute Force Trigger Is Two Steps

What happens: You look up how to write a trigger and it looks very different from a regular query.

Why it happens: PostgreSQL triggers require a FUNCTION to be written first. The trigger then calls the function.

Fix: Write these two blocks in order:

```
CREATE OR REPLACE FUNCTION check_brute_force()
RETURNS TRIGGER AS $$
BEGIN
  IF (SELECT COUNT(*) FROM events WHERE user_id = NEW.user_id
      AND event_type = 'login_failed'
      AND timestamp > NOW() - INTERVAL '15 minutes') >= 5 THEN
    INSERT INTO alerts (event_id, alert_type, severity, status, created_at)
    VALUES (NEW.event_id, 'brute_force', 'High', 'Open', NOW());
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER brute_force_trigger
AFTER INSERT ON events
FOR EACH ROW
EXECUTE FUNCTION check_brute_force();
```

Difficulty 3 — Integration Breaking on Day 15

What happens: Fatima changes her Users table column from user_name to username two days ago but did not tell Usman. Now Usman's Events API that JOINs Users is broken.

Fix: The rule is simple. After Day 6, no one changes their column names without posting in the WhatsApp group first. Add this as a team rule tomorrow.

Difficulty 4 — Sessions last_active Not Updating

What happens: The Sessions Monitor page shows sessions as active for 10 hours because last_active is never updated.

Fix: In every API endpoint that processes a user action, Fatima adds one line after the main logic: UPDATE sessions SET last_active = NOW() WHERE session_token = current_token.

Difficulty 5 — Forgetting org_id Filters

What happens: The Events Log page shows events from all organizations instead of just the logged-in organization.

Why it happens: The API query does SELECT * FROM events without WHERE org_id = current_user_org.

Fix: Every single query in the system must filter by org_id. Build this habit from Day 6. Before every query ask: am I filtering by org_id?

Difficulty 6 — Seed Data Takes Long

What happens: On Day 9 you realize you have no test data and the dashboard shows empty tables.

Fix: On Day 9, each person inserts at least this much data for their own tables: 2 organizations, 5 users per org, 3 roles, 3 assets per org, 3 sessions, 50 events with a mix of login_success and login_failed, 5 threat intelligence IPs.

For the brute force demo to work you need at least 5 login_failed events for the same user within 15 minutes. Insert these manually or use the Simulate Attack button.

Difficulty 7 — Chart.js Not Showing Data

What happens: The statistics page shows an empty chart.

Why it happens: The data format returned by the API does not match what Chart.js expects.

Fix: Chart.js needs two arrays: one for labels (dates) and one for values (counts). The API should return something like: { labels: ['2024-01-01', '2024-01-02'], data: [12, 34] }. Abdullah must format the SQL query result into this shape in the API.

12. DBMS Concepts You Will Demonstrate

This section is for the report and for answering instructor questions. Each concept is shown in ThreatLens in a specific way.

Concept	How ThreatLens Shows It	Who Demonstrates It
ER Model	8 entities with 1:N and M:N relationships, all drawn on draw.io with cardinality labels	Usman presents
Normalization 3NF	Each table has no repeating groups (1NF), no partial dependencies (2NF), no transitive dependencies (3NF). Documented for each table.	All Three
SQL Queries	5 threat detection queries using GROUP BY, HAVING, JOINs, EXTRACT, COUNT DISTINCT, subqueries	All Three
Indexes	4 indexes on Events table for timestamp, user_id, ip_address, org_id. Shown with EXPLAIN output before and after.	Usman presents
Transactions (ACID)	BEGIN; INSERT INTO events; INSERT INTO alerts; COMMIT; — if alerts insert fails, events insert also rolls back	Usman presents
Triggers	After each INSERT into Events, the brute force trigger function automatically checks and inserts into Alerts if threshold hit	Usman presents
Stored Procedures	Brute force and off-hours queries saved as stored procedures. Called by the Flask API with CALL procedure_name()	Usman presents
Concurrency Control	Two people send events at the same time. PostgreSQL handles isolation. No data loss or duplicate keys.	Fatima, Abdullah present
RBAC	Analyst role can view alerts but cannot change system settings. Admin can do everything. Enforced in both API and database queries.	Fatima presents
Query Optimization	EXPLAIN run on brute force query before and after adding indexes. Response time compared.	Usman presents

13. Quick Reference — Check This Every Day

6 Dates That Cannot Be Missed

Day	What Must Be Done
Day 2	JIRA set up, all tasks added, everyone has GitHub cloned and tools installed
Day 5	All 8 CREATE TABLE statements running in PostgreSQL. Tables visible in pgAdmin.
Day 10	All 5 threat detection queries tested and working. All API endpoints returning data from Postman.
Day 15	Full integration done. DB + API + Dashboard all connected on Usman's shared database.
Day 20	All testing complete, all bugs fixed, full demo rehearsed once.
Day 24	Final report merged, formatted, and ready to submit.

What Done Means

A task is only done when all three of these are true:

34. It works correctly and you tested it yourself.
35. The code is committed and pushed to GitHub.
36. The JIRA task is dragged to Done.

If You Are Stuck

- Copy the exact error message. Google it. Check Stack Overflow.
- Still stuck after 30 minutes? Post the error in the WhatsApp group immediately.
- Do not spend more than 1 hour on the same error alone.
- Do not change another person's table or code without asking them first.

Three Team Rules — Agree on These Tomorrow

37. Everyone writes SQL, everyone writes Python, everyone writes HTML. No one is stuck in only one layer.
38. Column names are frozen after Day 6. No changes without telling the group first.
39. One Sunday check-in call every week. Non-negotiable.